Dessiner un graphe directement en espace écran

Jalon 1 : où en est le projet

Melissa Colin Quentin Lopez Yani Haouchine

6 novembre 2026

ENSEIRB-MATMECA 3A, projet semestriel · Encadrant : Romain Giot (LaBRI)

Plan

1. Le sujet en une slide
2. Un rapide état de l'art
3. Notre idée générale
4. Premiers choix techniques : les données, l'architecture, la loss
5. Premiers résultats
6. Idées pour la suite

Le sujet

Optimiser la position des nœuds d'un graphe directement en espace écran, avec des coûts calculés sur le dessin rendu en pixels.

ENTRÉE

Un graphe

arêtes, taille des nœuds, en option largeur des arêtes et couleurs

SORTIE

Des positions

(x, y) pour chaque nœud, et éventuellement l'image

MOYEN

Un rendu différentiable

nœuds et arêtes dessinés avec des SDF, d'où des coûts écran dérivables

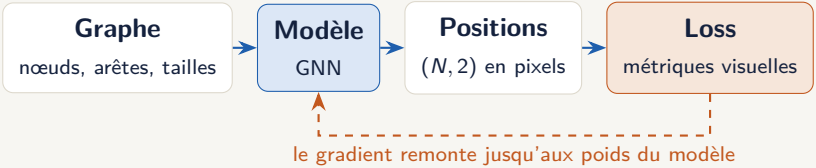
Question centrale du sujet : des coûts définis à l'écran donnent-ils des dessins plus fidèles au rendu final que des coûts géométriques ?

Un rapide état de l'art

Méthode	Principe
Kamada-Kawai, stress	optimisation classique, refaite pour chaque graphe
$(GD)^2$	descente de gradient sur des critères esthétiques, graphe par graphe
DeepGD	un GNN appris une fois, entraîné sur le stress
SmartGD	un GAN qui imite des dessins de référence
CoRe-GD (ICLR 2024)	un GNN récurrent qui se recâble selon son propre dessin ; meilleur stress sur Rome

Toutes optimisent dans un espace géométrique, où un nœud est un point et une arête un segment sans épaisseur. Les pixels n'arrivent qu'à l'affichage.

Notre idée : un réseau qui place les nœuds directement en pixels



Les trois axes du sujet, et où nous en sommes :

- des coûts calculés à l'écran : en cours, c'est notre loss ;
- l'optimisation directe, graphe par graphe : pas encore commencée ;
- un modèle appris sur la base Rome : notre point de départ, sans dessins de référence puisque la loss note le dessin produit.

Les données

Base Rome
11 534 graphes de 10 à 110 nœuds.
Découpage fixe : 70 % entraînement, 10 % validation, 20 % test.

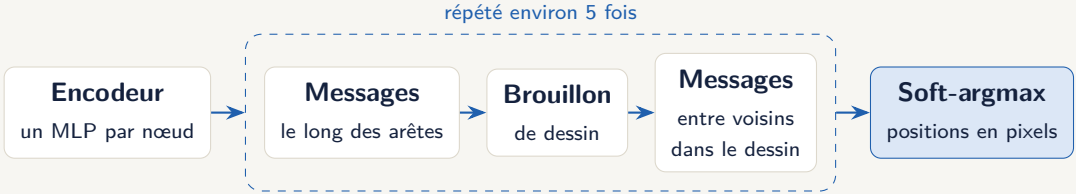
Ce que le modèle voit d'un nœud
Son degré, un bruit aléatoire et 8 encodages de position (vecteurs propres du Laplacien), plus les distances dans le graphe pour la loss.

Format PyTorch Geometric : plusieurs graphes traités en un seul batch, entraînement sur GPU.



La loss n'a pas besoin de dessins de référence. On pourrait donc générer des graphes aléatoires à volonté, au lieu de dépendre de la base Rome.

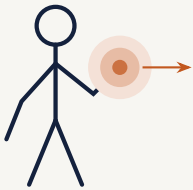
L'architecture : un GNN récurrent et une sortie en pixels



- Boucle reprise de CoRe-GD : deux nœuds proches dans le dessin échangent des messages, et peuvent apprendre à s'écarter.
- Soft-argmax en sortie : la position est la moyenne d'une carte de probabilités sur les pixels, donc toujours dans le cadre (slide suivante).

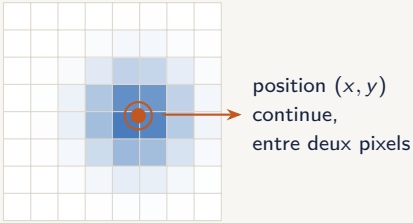
Soft-argmax : une sortie dans l'espace des pixels

Origine : la pose humaine



carte de chaleur d'une articulation
$\rightarrow$ sa position dans l'image

Chez nous : un nœud du graphe



probabilité par pixel
(softmax)

$$(x, y) = \sum_{\text{pixels } p} \text{softmax}(s)_p \cdot \text{coord}(p)$$

moyenne pondérée : différentiable, contrairement à argmax

Integral Pose Regression (Sun et al., 2018) · DSNT (Nibali et al., 2018)

La loss : les métriques de la littérature

La loss est une somme pondérée de métriques différentiables : $\mathcal{L} = \sum_k w_k \, m_k$. Nous avons d'abord trié les métriques existantes.

À conserver ou adapter

Stress de Kamada-Kawai	distances
Aspect ratio	proche d'un carré
Angular resolution	pas d'angle trop petit
Edge crossings	moins de croisements

Non essentiel pour nous

Cluster overlap	séparer les clusters
Neighborhood preservation	voisins conservés

La loss : les métriques choisies pour les nœuds

Stress

$d_G(u, v) = 3$ arêtes

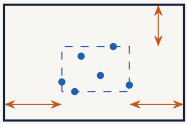


distance dessinée

$$\sum_{u<v} \frac{(\|p_u - p_v\| - d_{uv})^2}{d_{uv}^2}$$

respecter la structure du graphe

Écran

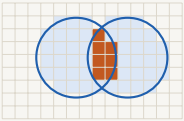


cadre $W \times H$

$$2 - \frac{\Delta x}{W} - \frac{\Delta y}{H}$$

occuper tout le cadre (inspirée de l'aspect ratio)

Chevauchement



pixels dans les deux disques

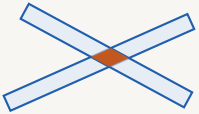
$$\sum_{\text{pixels}} \sigma\left(\frac{-\text{SDF}_i}{s}\right) \sigma\left(\frac{-\text{SDF}_j}{s}\right)$$

des nœuds, pixel par pixel avec des SDF

Prochaine étape : mettre le stress à l'échelle des pixels, il compte aujourd'hui une arête pour un pixel.

La loss : les métriques pour les arêtes, en cours

Chevauchement des arêtes



Pénalise la zone où deux arêtes épaisses se recouvrent, avec des SDF.

inspirée du nombre de croisements

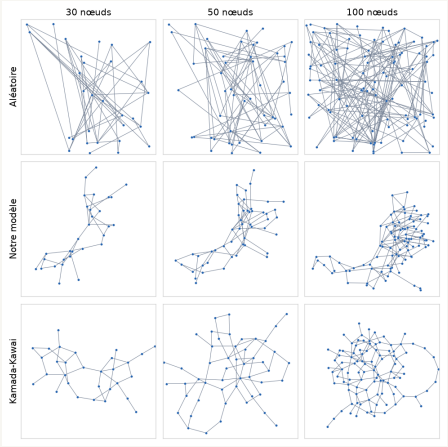
Angles entre arêtes



Éviter les angles trop petits entre arêtes d'un même nœud.

reprendre l'angular resolution telle quelle ? à décider

Premiers résultats : entre l'aléatoire et Kamada-Kawai



	Stress normalisé	Nœuds qui se chevauchent
Aléatoire	0,42	0,09 %
Notre modèle	**0,14**	**0,30 %**
Kamada-Kawai	0,06	0 %

Moyennes sur 300 graphes de test (stress : plus bas = mieux).

- La structure est bien meilleure qu'au hasard, mais pas encore au niveau de Kamada-Kawai.
- Au-delà de 50 nœuds, le modèle forme des amas, d'où plus de chevauchements qu'au hasard.
- Ce modèle a été entraîné avec une loss provisoire, avant la loss commune.

Suite : d'ici le jalon 2

1. Entraîner le modèle avec la loss commune, le stress mis à l'échelle des pixels
2. Ajouter les métriques sur les arêtes (chevauchements, angles)
3. Faire occuper tout le cadre aux dessins, qui restent souvent dans un coin
4. Stabiliser l'entraînement, dont la validation varie beaucoup d'une epoch à l'autre
5. Comparer aléatoire, Kamada-Kawai, méthodes apprises et notre modèle sur les mêmes métriques

Suite : alléger la sortie soft-argmax

Aujourd'hui : 250 000 scores par nœud (une carte 500×500), donc 4 graphes par batch au plus sur notre GPU.

Tête séparable
500 scores pour x, 500 pour y : même position, 250 fois moins de mémoire

Grille plus grossière
100×100 : la position reste continue

Précision mixte
calculs en 16 bits

Accumulation
simuler un batch de 16 avec 4×4

Suite : une taille d'image variable

Aujourd'hui, la taille de l'image est figée dans les poids : la tête a exactement $W \times H$ sorties.

Grille normalisée
Le modèle dessine dans $[0,1]^2$, puis on convertit en pixels à la sortie :
$x_{\text{px}} = m + x \cdot (W - 2m)$.
La taille de l'image devient une entrée, plus un hyperparamètre.

Dire au modèle pour quelle image il dessine
Lui donner le rapport W/H et la taille des nœuds rapportée à l'image (r/W), et tirer des tailles d'image variées pendant l'entraînement.

Option plus ambitieuse : un réseau qui donne un score pour n'importe quel point (x, y), évalué sur la grille voulue. Plus souple, mais il faut un calcul par nœud et par pixel.

Suite : rendre le modèle attentif aux arêtes

Les arêtes ne sont pas prédites, ce sont des segments entre les nœuds. Pourtant, la plupart des défauts à l'écran viennent d'elles.

A. Tête auxiliaire
Une tête en plus prédit les défauts de chaque arête (croisements, nœud caché, longueur). Entraînée par une perte auxiliaire, elle force le modèle à encoder l'information sur les arêtes.

B. Raffinement itératif
Une première passe propose des positions. On calcule exactement les défauts de chaque arête, puis une seconde passe les reçoit en entrée et corrige.

Bonus : affiner le modèle selon nos préférences

On annote des paires de dessins (« je préfère A à B »), on apprend un critique sur l'image (un CNN), puis on affine le modèle pour lui plaire, en le gardant proche du modèle de départ pour qu'il ne triche pas.

OPTION 1

RL

PPO ou REINFORCE : on tire des dessins, le critique les note, comme le RLHF

OPTION 2

DPO

directement à partir des paires, sans critique ni boucle de RL

OPTION 3

Rétropropagation directe

si critique et rendu sont différentiables (type DRaFT)